Lab 1

ece 260c. **Lab 1** Design Space Exploration

Shijie Sun / ysyx_25050135
Please enter your name and PID above– you agree to the academic integrity and course policies outlined in the Syllabus.

This lab will use the same Docker container as before. **Please ensure GUI is enabled** - see the Software Setup Guide if you are unsure.

GitHub Classroom for accessing all necessary data files and submitting your work.
Click here to unlock the assignment in GitHub Classroom.

Then run,

```
gh repo clone ABKCourses/ece260c-lab1-YourUsername
cd ece260c-lab1-YourUsername
```

Section I Synthesis & Technology Mapping

Synthesis is the process of transforming your RTL design (in our case, written in Verilog) into a gate-level netlist that is suitable for physical implementation. In ECE 260B, you may have had limited exposure to synthesis tools like Synopsys Design Compiler. You likely encountered typical inputs —including the Verilog RTL, a PDK-specific Liberty file, and an SDC (Synopsys Design Constraints) file —and outputs such as the synthesized Verilog netlist and estimated PPA (Performance, Power, Area) metrics.

In this section, we'll explore the synthesis tool Yosys, focusing on how circuit representations evolve from your original RTL input to the final gate-level netlist. Along the way, we'll examine some of the complexities involved in synthesizing real-world designs for real-world process technologies —and how various options within the tool can help you tune and improve your Quality of Results (QoR).

Starting with Yosys

Yosys is the open-source synthesis tool that is commonly paired with OpenROAD. While it does not support the full feature set and QoR expected of commercial tools, it is rapidly evolving and improving. Yosys is also widely adopted in the FPGA domain, with some FPGA vendors – such as Quicklogic and Cologne Chip – even using it as their primary synthesis tool.

Yosys is already a part of the `ece260c-essential` *Image* / Docker image you have used in Lab 0. OpenROAD-flow-scripts invokes Yosys as the primary tool in its Synthesis step.

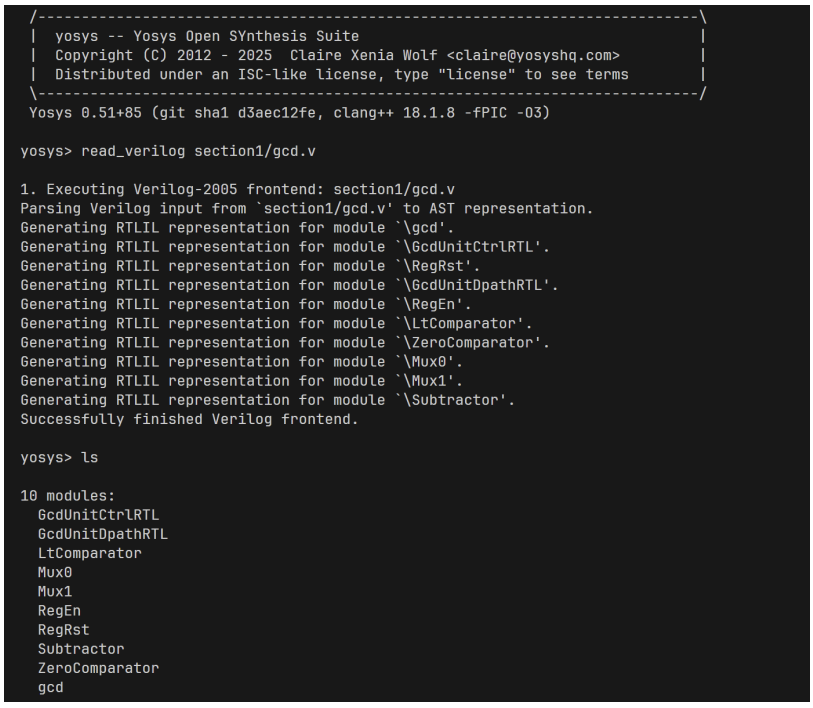
Yosys can be started without OpenROAD or ORFS, however, using t he `yosys` command. Before running this command, `cd` into the `section1/` directory. Then, run `yosys` – you should see a license/version notice and then the `yosys>` interactive prompt.

Yosys stores your design in-memory as a series of modules. When you open it, the session starts with an empty design. Here, we will be using the included `section1/gcd.v`, adapted from ORFS. **Run the following command within yosys to load the design:**

```
read_verilog section1/gcd.v
```

You can verify whether the design is loaded by running `ls` to list the loaded modules.

Q1.1 Paste a screenshot below of the `ls` command run while `gcd` is loaded.



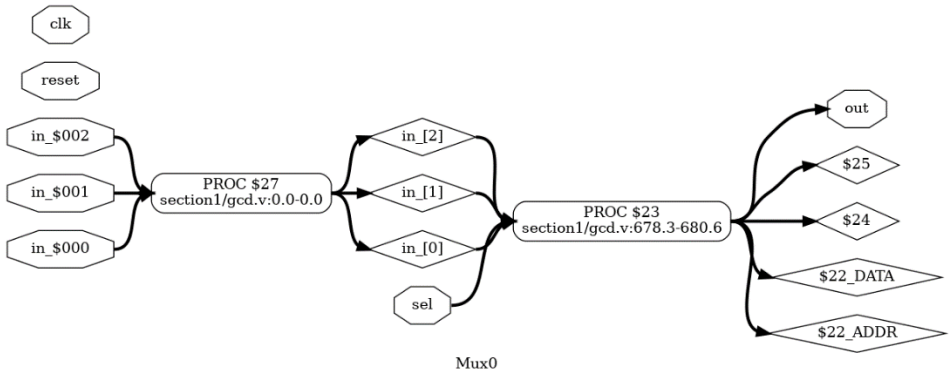
Before diving into specific commands, it is important to understand the role of Design Space Exploration (DSE) in the synthesis flow. A typical DSE involves evaluating multiple architectural and implementation options to optimize for various metrics such as area, timing, power, or performance. It helps designers better understand trade-offs, identify bottlenecks, and make informed decisions to improve the overall Quality of Results (QoR). Now that the modules have been loaded into memory, we can begin exploring the design using several key Yosys commands – including, **but not limited to** , the following:

Command	Description
`cd ..` `cd <module>`	Allows you to step inside a module in the design (A convenient version of `select`)
`ls` `ls <module>`	When outside any module, lists modules When inside a module (using cd), lists instances and ports.
`select *` `select <query>`	Makes it possible to select a subset of logic by object name(s) or by advanced functions like logic cones
`show` `show <module>`	Renders the design to a graph and displays it in a new graphical window
`stat` `stat <module>`	Without any flags, shows counts of specific cells
`dump` `dump <module>`	Dumps the selected module out as RTLIL, the intermediate textual representation Yosys uses.

We can now start to understand how the design flows through the synthesis tool. First, Run the following: `show Mux0`
A new GUI window should open with the schematic for the Mux0 module – you may need to click the zoom-to-fit button.

Note: you may see errors in the terminal while `show` is running, even after it has closed. You can hit the enter key to return to the yosys prompt if this happens.

Q1.2 Paste a screenshot of the schematic. What elements are visible?



In the elements section of the schematic, various types of signals and components within the module are displayed. These include input signals such as clk, reset, sel, and the port signals in_$000, in_$001, and in_$002. The output signal out is also shown. Additionally, there is an intermediate array signal named in_, as well as several temporary internal signals automatically generated by Yosys, such as $22_DATA, $22_ADDR, $24 and $25. These additional signals are introduced by Yosys during the synthesis process to translate array indexing and multiplexer logic into low-level gate-level structures. They are part of the tool's internal representation and are expected during the transformation of high-level constructs.

Processes

In Verilog, a process refers to a procedural block that executes in response to certain events, which may be explicitly or implicitly defined. You should be familiar with the following:

- `always @*` / `always_comb` / `always_latch` blocks – which execute whenever a signal the block depends on changes
    - `always_comb` / `always_latch` are SystemVerilog features that combat always @*'s ambiguity by helping ensure correct intent, such as whether the block should infer combinational logic or latches.
- `always @(posedge clk)` or `always @(negedge clk)` blocks – which execute on an edge, typically used for clocks or IOs. They're typically used to describe sequential logic.
- `initial` blocks – which run once at the start of a simulation. While primarily used for testbenches and initialization in simulation, some FPGA platforms may reinterpret them as actions triggered on reset.

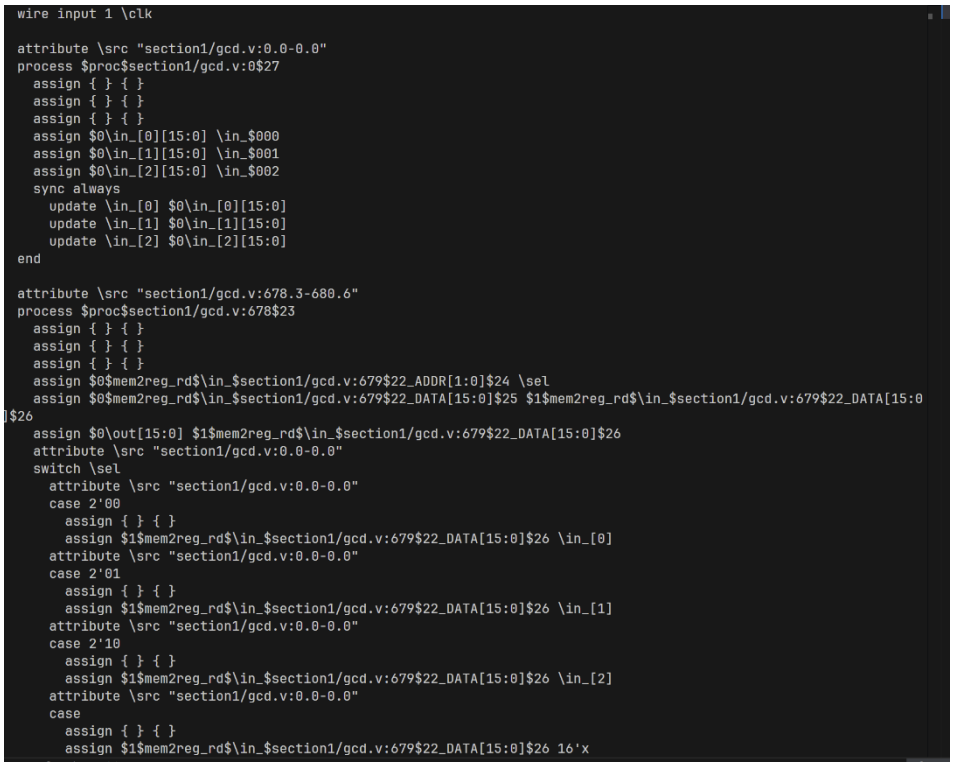
The process model is advantageous because it aligns naturally with event-driven simulator architectures, while also being synthesizable into either pure combinational logic or **stateful** logic implemented using flip-flops or latches.

What you will notice in the schematic of `Mux0` is that it is challenging to discern the behavior because processes are opaque i.e., they are not broken down into logic gates or control flow.

When Yosys transforms the design, it adds constructs that are not easily expressible in Verilog, like explicit latches. That's why, when you want to export or view Yosys's full database, the resulting code is provided in RTLIL (Register Transfer Level Intermediate Language) – an intermediate representation language used by Yosys during synthesis. It provides a low-level, implementation-friendly view of a Verilog design by translating behavioral and structural constructs into a flattened set of logic primitives, wires and cells. This format is useful for understanding exactly how high-level Verilog maps to synthesizable logic.

Q1.3 Run the following code and paste a screenshot of the resulting RTLIL. Then, in 2-3 sentences, compare the RTLIL to the Verilog source for `Mux0` in `gcd.v`

```
dump Mux0
```



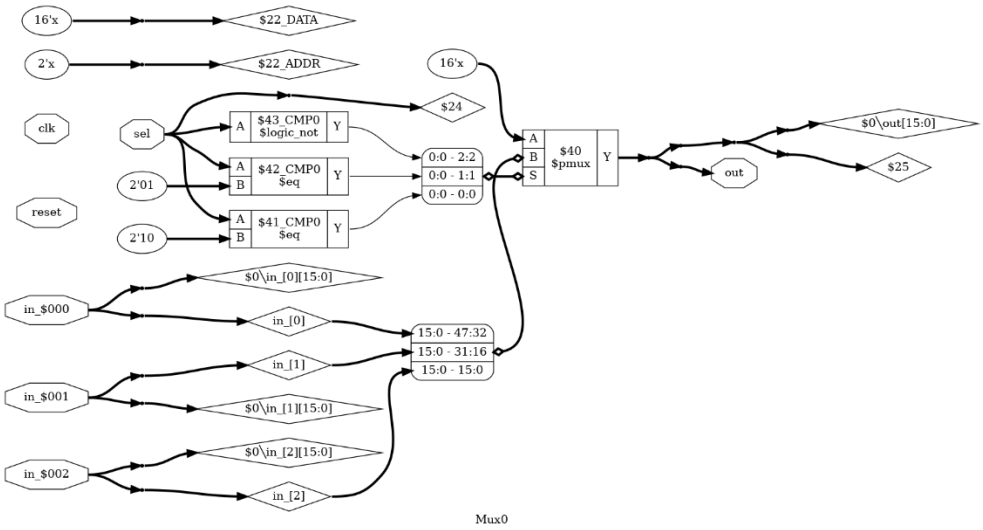
```
wire input 1 \clk

attribute \src "section1/gcd.v:0.0-0.0"
process $proc$section1/gcd.v:0$27
  assign { } { }
  assign { } { }
  assign { } { }
  assign $0\in_[0][15:0] \in_$000
  assign $0\in_[1][15:0] \in_$001
  assign $0\in_[2][15:0] \in_$002
  sync always
    update \in_[0] $0\in_[0][15:0]
    update \in_[1] $0\in_[1][15:0]
    update \in_[2] $0\in_[2][15:0]
end

attribute \src "section1/gcd.v:678.3-680.6"
process $proc$section1/gcd.v:678$23
  assign { } { }
  assign { } { }
  assign { } { }
  assign $0$mem2reg_rd$\in_$section1/gcd.v:679$22_ADDR[1:0]$24 \sel
  assign $0$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$25 $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0
]$26
  assign $0\out[15:0] $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$26
  attribute \src "section1/gcd.v:0.0-0.0"
  switch \sel
    attribute \src "section1/gcd.v:0.0-0.0"
    case 2'00
      assign { } { }
      assign $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$26 \in_[0]
    attribute \src "section1/gcd.v:0.0-0.0"
    case 2'01
      assign { } { }
      assign $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$26 \in_[1]
    attribute \src "section1/gcd.v:0.0-0.0"
    case 2'10
      assign { } { }
      assign $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$26 \in_[2]
    attribute \src "section1/gcd.v:0.0-0.0"
    case
      assign { } { }
      assign $1$mem2reg_rd$\in_$section1/gcd.v:679$22_DATA[15:0]$26 16'x
```

The Verilog source for Mux0 in gcd.v uses high-level behavioral constructs like always @(*) and array indexing to describe functionality concisely. In contrast, the RTLIL version flattens this logic into low-level primitives and explicit wiring, exposing th e synthesized structure and making implicit behaviors, like multiplexing or inferred latches, fully explicit.

Let us now map processes to real logic: Run the `proc` command to convert all processes in the design to a circuit representation. Then, run `show Mux0` again.

Q1.4 In 1-2 sentences, what can you see now in `Mux0`? Does it match the behavior described in Verilog in section1/gcd.v?

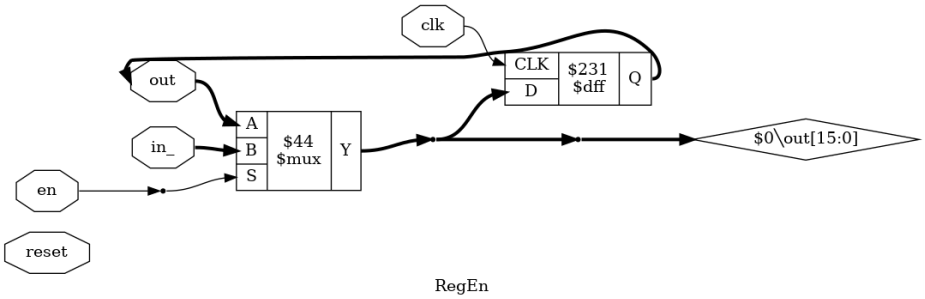
Now, in the Mux0 module, I can clearly see the inputs and outputs, as well as the interconnecting wires, logic gates, and multiplexers. This gate-level representation corresponds well with the behavior described in the original Verilog code in section1/gcd.v.



Mux0

Now, let's try sequential (stateful) logic. Run `show RegEn`.

Q1.5 Paste a screenshot. In 2-3 sentences, what instances are present? What do you notice about how the always block in the original Verilog was converted?



RegEn

In the schematic, there is one $mux instance and one $dff instance, indicating that the always block was translated into a register with conditional input. Notably, the reset signal is not connected or used, which means the original Verilog did not specify reset behavior, and thus no reset logic was synthesized.

The Synthesis Flow

With an understanding of processes, we can now continue the synthesis process.

The **first step** in synthesis is to tell Yosys the **hierarchy** of the design – the relationship between modules based on how they instantiate one another. The top module serves as the **root** of this hierarchy; it is not instantiated by any other module and typically represents the top-level block or chip being synthesized.
Run the following to set the top module to gcd

```
hierarchy -top gcd
```

Typically, before synthesis, we would execute `flatten` to flatten the netlist before performing optimizations, as optimizations are conservative and do not cross module barriers.
To keep the schematic from exploding in size, however, we will do this later. Let us perform a multi-module synthesis with optimization as follows:

```
opt
synth
stat
```

For your reference, you can look at the yosys references for opt and prep (which combines several optimization passes).

Q1.6 Paste a screenshot of the `stat` command run above. In 1-2 sentences, what do you notice is different in terms of cell composition compared to what you saw in the schematic you talked about in Q1.4?

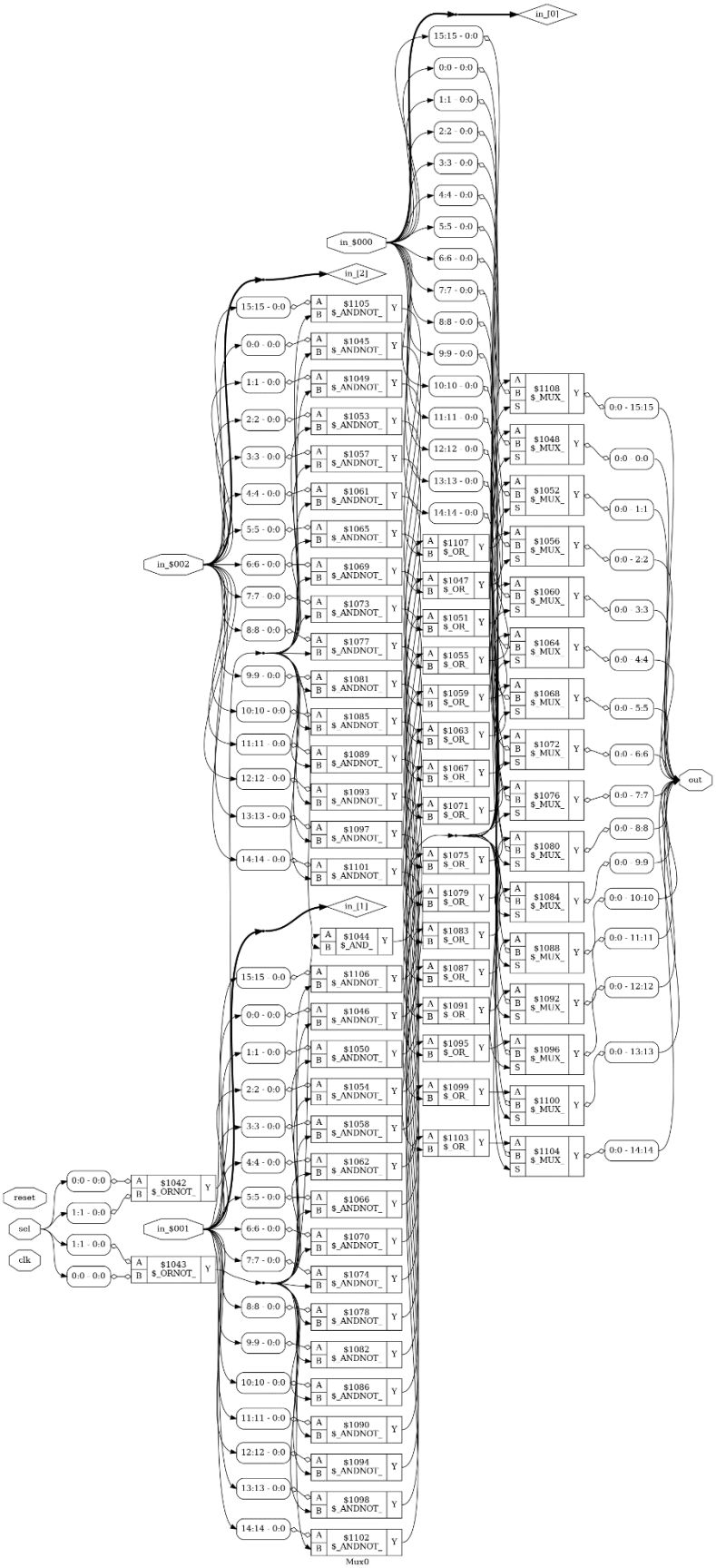
```
=== Mux0 ===

   Number of wires:                 61
   Number of wire bits:            167
   Number of public wires:          10
   Number of public wire bits:     116
   Number of ports:                  7
   Number of port bits:             68
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 67
     $_ANDNOT_                      32
     $_AND_                          1
     $_MUX_                         16
     $_ORNOT_                        2
     $_OR_                          16
```

After synthesis, the cell composition has changed significantly: complex constructs like the $pmux, $eq and etc. have been broken down into many low-level primitives, such as $_ANDNOT_, $_MUX_, and $_OR_, reflecting fine-grained logic gates optimized for implementation.

Primitives & The Internal Cell Library

Q1.7 Run show Mux0 again and paste a screenshot here (you may need to zoom in). In 2-3 sentences, what do you notice is different from the previous versions of Mux0 (like in Q1.4) in terms of cells utilized and the structure of the circuit network?
Hint: you can get a general idea of logic flow by looking from left to right (inputs on left, first stage next, .., outputs on right). `$_ANDNOT_` implements the function `Y = A & ~B` and `$_ORNOT_ = Y = A | ~B`. A suffix

The circuit implementation is now much more concrete and precise, operating at the bit level. Instead of high-level constructs like a single multiplexer, it uses lower-level, technology-mapped cells such as $_ANDNOT_, $_ORNOT_, and $_MUX_ to explicitly implement logic operations. These cells work together to process individual input bits (in_$000, in_$001, in_$002, etc.) and the sel signal in a staged and structured manner.


reset
sel
clk
in_$001
in_$002
in_$000
in_[2]
in_[1]
in_[0]
out
Mux0

What you saw above is the evolution of the logic database through two different stages of what Yosys calls its "Internal Cell Library". Internal Cells, also called *Primitives*, are powerful technology-agnostic building blocks used to represent a design in a modular and abstract form. In Yosys, your input Verilog is first instantiated into a modular netlist containing a mix of processes and what are known as "word-level" internal cells that correspond directly to Verilog's (multi-bit) mathematical-logical operations (i.e. a 32-bit AND, a 16-bit subtract, or a 4-bit DFF) and even other high-level constructs such as FSMs or memory blocks (for which Yosys contains several extraction tools to lift out). When the synth command is called, Yosys lowers these word-level internal cells to gate-level cells – logic primitives that more closely resemble the elements found in real PDKs, such as AND gates, muxes, and DFFs.

In the `section1/sub` folder, you will ask Yosys to write out the post-synthesis netlist that is currently loaded in both Verilog and RTLIL. Assuming you're in the `section1/` folder, do this by running :

```
write_verilog sub/postsynth.v
write_rtlil sub/postsynth.rtlil
```

Confirm that both files were created successfully. Verify that the files are present and non-empty. **You may wish to commit at this stage as a way to backup your progress.**

Optimization and Techmapping

So far, our design has remained in the technology-agnostic domain. We need to transform it to use our PDK's standard cells – techmapping. A naive techmapping is possible – the common cells in the Internal Library can usually be mapped 1:1 to PDK cells. This would, however, leave performance on the table as optimizations using the delay data of PDK cells is possible, and these optimizations involve analyzing several possible representations for a given circuit and its subcircuits.

Yosys uses UC Berkeley's ABC, a synthesis tool capable of performing combinational and sequential logic optimization for both ASICs and FPGAs. When we call Yosys's `abc` command, it translates your design into an ABC-compatible format – optionally splitting it into combinational sections – then calls ABC to optimize the logic, and finally maps the results back into Yosys's internal representation. While ABC can be scripted and run independently, Yosys provides sensible defaults that make this process efficient and easy to use, saving considerable time during synthesis.

Internally, ABC operates by converting your circuit into an AND-Inverter Graph (AIG) —a simplified combinational representation composed of two-input AND gates and inverters. AIGs are particularly well-suited for a variety of optimization tasks, including redundancy elimination, logic equivalence checking, and efficient search for optimized circuit structures. ABC can also perform retiming.

While ABC can be run in a technology-agnostic mode —optimizing logic purely at the primitive level (which Yosys's synth command already performs) —we will be using it with a Liberty file. A Liberty file provides detailed timing, power, and functional information for each standard cell in a given PDK (typically grouped by threshold voltage or cell style) – for a quick tutorial check here. ABC uses this information to map portions of the logic (called "cuts") to optimal standard cells, continuing this process until the entire design has been transformed. This technology mapping can be guided by constraints such as timing or area, enabling more efficient and physically-aware synthesis. OpenROAD can also call into ABC at runtime to re-optimize the design based on updated constraints – this is part of the rmp module.

Now, run the following script to clear out the design, synthesize it, and **finally techmap it with the IHP 130 PDK** (a copy of the liberty file is included in your repo):

```
design -reset # Clear out the loaded design
read_verilog gcd.v
hierarchy -top gcd

flatten
# Technology-agnostic Verilog-level optimizations
prep
# Synthesize to technology-agnostic gate-level primitives
synth
# Replace any agnostic DFF primitives that cannot be mapped directly to IHP
130's DFF cells
dfflibmap -liberty sg13g2_stdcell_typ_1p20V_25C.lib
# Call into ABC to synthesize with a timing goal of 4000 ps = 4 ns  on the
worst path.
abc -D 4000 -liberty sg13g2_stdcell_typ_1p20V_25C.lib
# Remove disconnected wire that ABC optimized away
opt_clean
```

Q1.8 Run the following and paste a screenshot here. You should be able to see the Area statistics.

stat -liberty `sg13g2_stdcell_typ_1p20V_25C.lib`

```
yosys> stat -liberty section1/sg13g2_stdcell_typ_1p20V_25C.lib

29. Printing statistics.

=== gcd ===

   Number of wires:                327
   Number of wire bits:            960
   Number of public wires:         102
   Number of public wire bits:     735
   Number of ports:                  8
   Number of port bits:             54
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                252
     $scopeinfo                     10
     sg13g2_a21o_1                   1
     sg13g2_a21oi_1                  6
     sg13g2_a221oi_1                16
     sg13g2_a22oi_1                 17
     sg13g2_and2_1                   2
     sg13g2_and4_1                   1
     sg13g2_dfrbp_1                 35
     sg13g2_inv_1                   18
     sg13g2_mux2_1                  32
     sg13g2_nand2_1                  8
     sg13g2_nand2b_1                10
     sg13g2_nand3_1                  5
     sg13g2_nor2_1                  11
     sg13g2_nor2b_1                 21
     sg13g2_nor3_1                  20
     sg13g2_nor4_1                   5
     sg13g2_o21ai_1                 12
     sg13g2_xnor2_1                 16
     sg13g2_xor2_1                   6

   Area for cell type $scopeinfo is unknown!

   Chip area for module '\gcd': 3974.707800
     of which used for sequential elements: 1651.104000 (41.54%)
```

For your reference, you can take a look at more advanced techmapping for memories, FSMs, or arithmetic units. Take a look at Yosys' extract functionality or how extract_fa is used to add full-adder cells available in some PDKs.

Timing and Power with OpenSTA

While stat gives you a cell-based area estimate, you likely also want tim ing and power information to fully evaluate your design. Unfortunately, this infrastructure is not yet part of Yosys so we will instead rely on OpenSTA

Before we can use OpenSTA, we need to export a techmapped netlist. You will also need to submit this, so: (i) run the following command (assuming you're in `./section1`) and then, (ii) after verifying that the file is not empty, exit yosys using the `exit` command:

```
write_verilog sub/postmap.v
```

Exiting Yosys , you can run OpenSTA by running`sta` in your terminal – it will also present an interactive CLI.

Q1.9 We will start with checking whether timing is met. Run the following and paste a screenshot below of the final timing report. In 1 -2 sentences, is timing met?

```
read_verilog sub/postmap.v
read_liberty sg13g2_stdcell_typ_1p20V_25C.lib
link_design gcd
# OpenSTA defaults to using the PDK's units loaded from the liberty file.
Try running report_units.
create_clock -name clk -period 4 {clk}
# Write a Standard Delay Format file as part of your submission
write_sdf sub/postmap.sdf
report_checks
```

```
sta [/work/ece260c-lab1-ShijieSun] report_checks
Startpoint: _432_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _451_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _432_/CLK (sg13g2_dfrbp_1)
   0.21    0.21 v _432_/Q (sg13g2_dfrbp_1)
   0.09    0.30 v _243_/Y (sg13g2_nor2b_1)
   0.13    0.42 ^ _290_/Y (sg13g2_o21ai_1)
   0.11    0.54 v _291_/Y (sg13g2_a21oi_1)
   0.14    0.68 ^ _294_/Y (sg13g2_o21ai_1)
   0.11    0.79 v _295_/Y (sg13g2_a21oi_1)
   0.16    0.95 ^ _298_/Y (sg13g2_o21ai_1)
   0.12    1.07 v _300_/Y (sg13g2_a221oi_1)
   0.15    1.22 ^ _301_/Y (sg13g2_nor3_1)
   0.08    1.30 v _303_/Y (sg13g2_nor3_1)
   0.14    1.44 ^ _304_/Y (sg13g2_nor3_1)
   0.09    1.52 v _306_/Y (sg13g2_nor3_1)
   0.14    1.66 ^ _307_/Y (sg13g2_nor3_1)
   0.13    1.80 v _340_/Y (sg13g2_nand3_1)
   0.47    2.27 ^ _382_/Y (sg13g2_a21oi_1)
   0.14    2.41 v _383_/Y (sg13g2_a22oi_1)
   0.14    2.55 ^ _385_/Y (sg13g2_a221oi_1)
   0.00    2.55 ^ _451_/D (sg13g2_dfrbp_1)
           2.55   data arrival time

   4.00    4.00   clock clk (rise edge)
   0.00    4.00   clock network delay (ideal)
   0.00    4.00   clock reconvergence pessimism
           4.00 ^ _451_/CLK (sg13g2_dfrbp_1)
  -0.14    3.86   library setup time
           3.86   data required time
---------------------------------------------------------
           3.86   data required time
          -2.55   data arrival time
---------------------------------------------------------
           1.31   slack (MET)
```

Yes, the timing is met (MET). The data required time is 3.86 ns, while the actual data arrival time is 2.55 ns, resulting in a positive slack of 1.31 ns. This means the data arrives early enough and does not violate the timing constraint.

While the report_checks output is useful when we have a specific clock frequency target in mind, we'll also want to determine the **maximum achievable performance** of our synthesized design —that is, the highest frequency it can reliably support based on current timing constraints. OpenSTA provides a convenient function for this: `report_clock_min_period`.

Q1.10 Run `report_clock_min_period` and paste a screenshot below.
In 3-4 sentences, explain how the minimum period or f_{max} compares to the 4000 ps target we asked ABC for above? (For those unfamiliar with f_{max}, please check here)
Does this performance, relative to the 4000 ps target, indicate anything about synthesis' effort i.e.

- Did ABC fail to meet the timing goal?
- Did ABC meet the timing goal and then stop?
- Did ABC meet the timing goal and then some?

Based on this, do you think it is possible to extract even more performance by resynthesizing with a lower clock period target? If so, how much lower would you try?

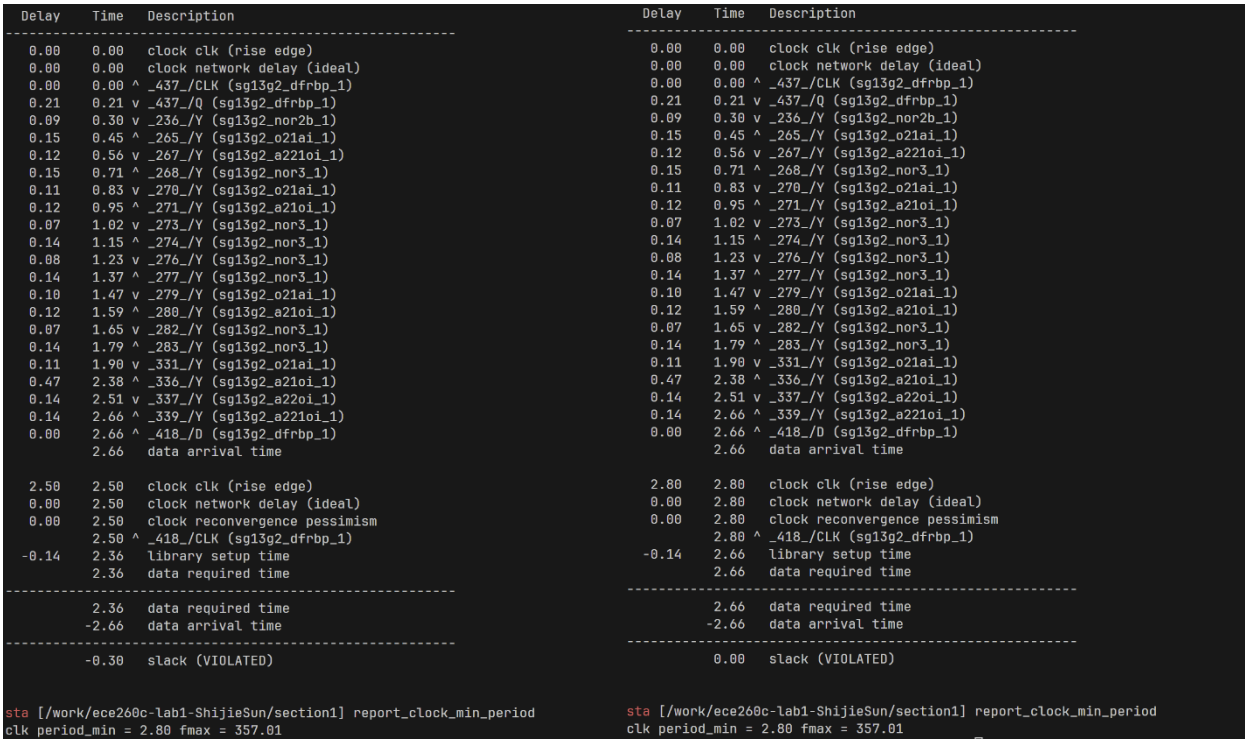
```
sta [/work/ece260c-lab1-ShijieSun] report_clock_min_period
clk period_min = 2.69 fmax = 371.19
```

The minimum clock period achieved is 2.69 ns, which is significantly better than the 4 ns target we originally asked ABC for. This corresponds to an f_max of approximately 371.19 MHz, meaning ABC not only met the timing goal but exceeded it by a good margin. This suggests that ABC had room to optimize further and did not just stop after meeting the 4 ns constraint. Based on this, it seems possible to extract more performance by resynthesizing with a tighter target —for example, trying a clock period around 2.5 ns or even 2.8 ns could be a reasonable next step to explore the performance limits.

I experimented with target clock periods of 2.5ns and 2.8ns, but both attempts led to timing violations. Surprisingly, the resulting period_min after optimization was worse than the initial default, stabilizing at 2.8ns in both cases, indicating no tangible benefit from the more aggressive constraints.

ABC in Yosys attempts to optimize the design to meet the target clock period specified with -D, but it does not guarantee success. If the circuit's inherent structure or the standard cells in the library cannot support tighter delays, ABC cannot achieve the desired timing. The optimization process is heuristic, not exhaustive, meaning it makes intelligent guesses rather than exploring

all possibilities. In some cases, overly aggressive timing constraints may lead ABC to apply transformations that increase complexity and actually worsen timing. This is why setting a smaller -D value (e.g., 2.0ns) might result in a worse clk period_min (e.g., 2.8ns). ABC's optimization behavior is also discrete —each -D value triggers a different optimization path, which may fall into different local optima. Therefore, the final timing result is not guaranteed to improve linearly with smaller target periods.



```
  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _437_/CLK (sg13g2_dfrbp_1)
   0.21    0.21 v _437_/Q (sg13g2_dfrbp_1)
   0.09    0.30 v _236_/Y (sg13g2_nor2b_1)
   0.15    0.45 ^ _265_/Y (sg13g2_o21ai_1)
   0.12    0.56 v _267_/Y (sg13g2_a221oi_1)
   0.15    0.71 ^ _268_/Y (sg13g2_nor3_1)
   0.11    0.83 v _270_/Y (sg13g2_o21ai_1)
   0.12    0.95 ^ _271_/Y (sg13g2_a21oi_1)
   0.07    1.02 v _273_/Y (sg13g2_nor3_1)
   0.14    1.15 ^ _274_/Y (sg13g2_nor3_1)
   0.08    1.23 v _276_/Y (sg13g2_nor3_1)
   0.14    1.37 ^ _277_/Y (sg13g2_nor3_1)
   0.10    1.47 v _279_/Y (sg13g2_o21ai_1)
   0.12    1.59 ^ _280_/Y (sg13g2_a21oi_1)
   0.07    1.65 v _282_/Y (sg13g2_nor3_1)
   0.14    1.79 ^ _283_/Y (sg13g2_nor3_1)
   0.11    1.90 v _331_/Y (sg13g2_o21ai_1)
   0.47    2.38 ^ _336_/Y (sg13g2_a21oi_1)
   0.14    2.51 v _337_/Y (sg13g2_a22oi_1)
   0.14    2.66 ^ _339_/Y (sg13g2_a221oi_1)
   0.00    2.66 ^ _418_/D (sg13g2_dfrbp_1)
           2.66   data arrival time

   2.50    2.50   clock clk (rise edge)
   0.00    2.50   clock network delay (ideal)
   0.00    2.50   clock reconvergence pessimism
           2.50 ^ _418_/CLK (sg13g2_dfrbp_1)
  -0.14    2.36   library setup time
           2.36   data required time
---------------------------------------------------------
           2.36   data required time
          -2.66   data arrival time
---------------------------------------------------------
          -0.30   slack (VIOLATED)


sta [/work/ece260c-lab1-ShijieSun/section1] report_clock_min_period
clk period_min = 2.80 fmax = 357.01
```

```
  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _437_/CLK (sg13g2_dfrbp_1)
   0.21    0.21 v _437_/Q (sg13g2_dfrbp_1)
   0.09    0.30 v _236_/Y (sg13g2_nor2b_1)
   0.15    0.45 ^ _265_/Y (sg13g2_o21ai_1)
   0.12    0.56 v _267_/Y (sg13g2_a221oi_1)
   0.15    0.71 ^ _268_/Y (sg13g2_nor3_1)
   0.11    0.83 v _270_/Y (sg13g2_o21ai_1)
   0.12    0.95 ^ _271_/Y (sg13g2_a21oi_1)
   0.07    1.02 v _273_/Y (sg13g2_nor3_1)
   0.14    1.15 ^ _274_/Y (sg13g2_nor3_1)
   0.08    1.23 v _276_/Y (sg13g2_nor3_1)
   0.14    1.37 ^ _277_/Y (sg13g2_nor3_1)
   0.10    1.47 v _279_/Y (sg13g2_o21ai_1)
   0.12    1.59 ^ _280_/Y (sg13g2_a21oi_1)
   0.07    1.65 v _282_/Y (sg13g2_nor3_1)
   0.14    1.79 ^ _283_/Y (sg13g2_nor3_1)
   0.11    1.90 v _331_/Y (sg13g2_o21ai_1)
   0.47    2.38 ^ _336_/Y (sg13g2_a21oi_1)
   0.14    2.51 v _337_/Y (sg13g2_a22oi_1)
   0.14    2.66 ^ _339_/Y (sg13g2_a221oi_1)
   0.00    2.66 ^ _418_/D (sg13g2_dfrbp_1)
           2.66   data arrival time

   2.80    2.80   clock clk (rise edge)
   0.00    2.80   clock network delay (ideal)
   0.00    2.80   clock reconvergence pessimism
           2.80 ^ _418_/CLK (sg13g2_dfrbp_1)
  -0.14    2.66   library setup time
           2.66   data required time
---------------------------------------------------------
           2.66   data required time
          -2.66   data arrival time
---------------------------------------------------------
           0.00   slack (VIOLATED)


sta [/work/ece260c-lab1-ShijieSun/section1] report_clock_min_period
clk period_min = 2.80 fmax = 357.01
```

We can also get a sense of power usage. OpenSTA provides the ability to run static power analysis, with support for loading VCD files from simulation (check the OpenSTA repo for implementation details).

Q1.11 Run `report_power` and paste a screenshot below:

```
sta [/work/ece260c-lab1-ShijieSun] report_power
Group                  Internal  Switching    Leakage      Total
                          Power      Power      Power      Power (Watts)
----------------------------------------------------------------
Sequential             5.19e-04   3.70e-05   1.82e-08   5.56e-04  71.5%
Combinational          1.48e-04   7.33e-05   2.90e-08   2.22e-04  28.5%
Clock                  0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Macro                  0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Pad                    0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
----------------------------------------------------------------
Total                  6.67e-04   1.10e-04   4.72e-08   7.78e-04 100.0%
                          85.8%      14.2%       0.0%
```

If you are interested, you can take a look at the OpenSTA manual.

Ensure you answered all questions in this section and ensure all the files you need for submission are in the `section1/sub` folder and that they are non-empty. Check the submission checklist in `section1/sub/README.md`.

Section 2 Design Space Exploration

Design Space Exploration (DSE) is the process of identifying key tunable parameters within a design and searching for optimal configurations based on performance, power, area, and other design constraints. In modern chip development, DSE is a continuous process that occurs at every level in the flow—from high-level architecture to low-level physical implementation. For example, microarchitects often write small simulation models to explore pipeline-level behavior and evaluate the performance of novel design ideas, particularly in comparison to previous generations within the same product family. These models are progressively refined as they undergo reviews and validation through various stages of the design cycle. In some cases, architects may even develop early RTL prototypes or simplified physical implementations to support their proposals with concrete data.

Physical Designers also have these tunable knobs. From what we covered in Week 1 Lecture 2 on floorplanning, you may remember that macros can be defined at different utilization levels, aspect ratios, or even standard cell types (i.e. low-Vt/high-density).

As chip design has grown increasingly complex and resource-intensive, the need to automate DSE has only become more critical. Automation approaches range from simple brute-force parameter sweeps to more advanced optimization techniques, such as machine learning–based tools like ORFS's AutoTuner, and even emerging deep learning methods.

However, tools like AutoTuner often require running the entire physical design flow, which can be time-consuming. In early-stage exploration, it's usually more practical to rely on simplified models or to run only a portion of the flow to generate quick performance, area, or power estimates. Once a promising design point is validated at an early stage, it can progress to later stages of refinement and eventually be evaluated through the full implementation flow.

Our Design

We will try out DSE by running a synthesis-only sweep on a design called ASQRT. This unit implements a configurable, parallel square root accelerator using an iterative algorithm to compute the square root of a 32-bit input integer—with each iteration of the algorithm generating or converging one bit of the output. The design includes `N_PIPES`, which controls the number of **parallel square root pipelines**. Each pipeline can be configured with a `N_DEPTH` parameter, which determines how many iterations are performed per cycle—effectively controlling how many cycles are required to compute the result (i.e., deeper pipelines reduce latency per bit). The `N_CYCLES` parameter sets the total number of cycles a pipeline is allowed to run. It must be greater than or equal to the number of output bits divided by the pipeline depth. **We will be running DSE over `N_PIPES`, `N_DEPTH`, and `N_CYCLES`.**

For this section, cd into the repo's `./section2` directory. You can take a look at the RTL in `section2/rtl`.
Also, we will need the matplotlib dependency to make graphs in Python. Install it by running:
`pip install matplotlib`

Our first step is to define certain design requirements – these will allow us to bound the range of our variables and reduce the number of variables.

We will start by establishing a performance requirement: we want a throughput of 1 square root per cycle. This gives us a 1:1 relationship between `N_PIPES` and `N_CYCLES`, i.e., if a single pipeline takes 2 cycles to compute a result, we will need 2 parallel pipelines to maintain our target throughput. At the end of this sweep, we will generate what can be referred to as a "frequency-independent iso-performance" comparison graph. The frequency-independent qualifier is key: while we are normalizing for throughput across configurations, the actual performance in terms of real-world throughput will still depend heavily on the maximum achievable frequency (f_{max}), which is why we will also analyze and plot f_{max}.

Q2.1 Knowing that each iteration of our square root operation produces one bit of the result, what is the minimum number of iterations required to compute the full correct output of the square root operation?
Hint: Consider the rounded-down square root of the largest possible 32-bit unsigned integer, and how many bits are needed to represent it.

The largest 32-bit unsigned integer is 4,294,967,295. The square root of this value, when rounded down, is 65,535. Since 65,535 requires 16 bits to represent in binary, the minimum

number of iterations needed to compute the full correct output of the square root operation is 16.
If the input is an n-bit number, then it requires n/2 iterations; if each iteration takes one clock cycle, then the computation will need n/2 clock cycles.

Q2.2 Knowing what we know about synthesis hierarchy and optimization, in what cases do you think the synthesis too l can or cannot correct for "overdesign" i.e. if we were to set N_DEPTH too high, such that there are redundant iterations that "do nothing" because the square -root result has already converged? Answer below in 4-5 sentences.

Synthesis tools can optimize away obvious redundant logic, such as constant propagation or dead code elimination. However, if the "overdesign" manifests as extra pipeline stages or iterations that may have functional significance depending on runtime inputs —even if the result has effectively converged —synthesis tools usually cannot remove them at compile time because they must maintain functional correctness for all inputs. Therefore, if N_DEPTH is set too high resulting in redundant iterations, the tool is unlikely to eliminate them automatically, leading to wasted area and increased latency. To address this, designers typically perform architectural optimizations or pruning before synthesis rather than relying on synthesis tools to fix overdesign.

Your answer to Q2.1 will help guide the definition of a **relationship between** `N_DEPTH` **and** `N_CYCLES`, and in turn, with `N_PIPES`. This enables us to **reduce our design search space from three variables to one** , simplifying our DSE process.
Q2.3 Express the mathematical relationship between N_PIPES with N_CYCLES and N_DEPTH with N_CYCLES.

N_PIPES = N_CYCLES
N_DEPTH = 16 / N_PIPES

We will continue to express this dependency explicitly in our Python code, which will drive DSE.

Scripting for DSE

Q2.4 Open `section2/explore.py` – this is our primary DSE script. Find and fill in the definitions for `N_DEPTH` and `N_PIPES` within the loop, including the resolution of the environment variables. Paste a screenshot of your updated code here:

section2 > explore.py

```python
import os
import json
import matplotlib.pyplot as plt

print("ECE 260C Lab 1: Design Space Explorer")
print("Shijie Sun / ysyx_25050135") # !!!

# Since we need an integer amount of iterating units, we should only use powers of two.
N_CYCLES_range = [16, 8, 4, 2, 1]

results = []

for N_CYCLES in N_CYCLES_range:
    N_PIPES = N_CYCLES          # TODO: Fill in here
    N_DEPTH = int(16 / N_CYCLES) # TODO: Fill in here

    print(f"== Running DSE step for {N_CYCLES} cycles - {N_PIPES} pipes at {N_DEPTH} depth ==")

    # Every step requires running yosys and OpenSTA.
    # We need to lift our parameters to environment variables so that Yosys can read in - corresponding to our parameters
    os.environ["DSE_N_CYCLES"] = str(N_CYCLES)
    # TODO: Add similar os.environ calls for N_PIPES and N_DEPTH...
    os.environ["DSE_N_PIPES"] = str(N_PIPES)
    os.environ["DSE_N_DEPTH"] = str(N_DEPTH)
```

For your convenience, the rest of the script has already been completed. Please review it to ensure you understand the flow: it calls Yosys and OpenSTA with different parameter combinations, generates synthesis and timing reports, and uses those reports to build a 2D plot for analysis.

Next, we will provide Yosys and OpenSTA with the necessary scripts —explore.py will call synth.tcl for Yosys and analysis.tcl for OpenSTA, respectively.

Q2.5 Open synth.tcl and follow the instructions to complete it. When it's done, screenshot it and paste it here. You will commit this file later.

section2 > synth.tcl

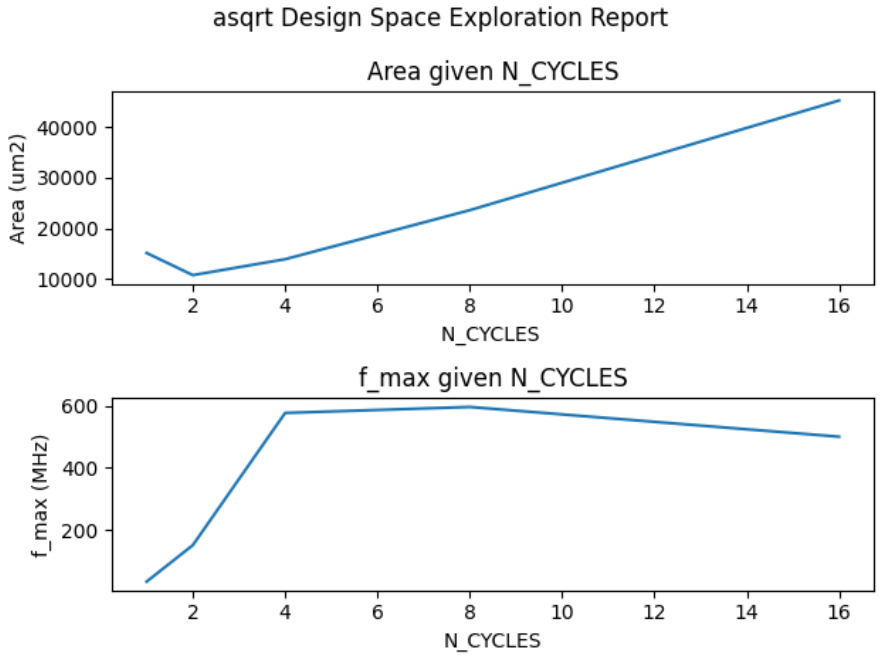
```
# ECE 260C Lab 1 - DSE
# Yosys Script

yosys -import

# You need to make sure this line outputs correctly, otherwise you will likely need to make fixes in explore.py
puts "Yosys is running for the design with $::env(DSE_N_CYCLES) cycles, $::env(DSE_N_PIPES) pipes, $::env(DSE_N_DEPTH) depth."

# Load our design, but don't flatten it or set hierarchy yet
read_verilog rtl/asqrt_top.v rtl/asqrt_pipe.v rtl/asqrt_iu.v

# chparam can be used to parametrize a module.
# When we use it for the top module like this, we can safely change the instantiation of the module itself
# without dealing with specific sub-instantiations.
# For this reason, it can be beneficial (when practical) to bring parameters up through to the top module (i.e. in asqrt_top.v)
chparam -set N_CYCLES $::env(DSE_N_CYCLES) asqrt_top
# TODO: the above chparam for N_PIPES and N_DEPTH
chparam -set N_PIPES $::env(DSE_N_PIPES) asqrt_top
chparam -set N_DEPTH $::env(DSE_N_DEPTH) asqrt_top

# Now, we set hierarchy
hierarchy -top asqrt_top
# TOOD: Here, insert the synthesis and techmapping script we used in Section 1, starting from the flatten command and ending wi
flatten
prep
synth
dfflibmap -liberty sg13g2_stdcell_typ_1p20V_25C.lib
abc -D 5000 -liberty sg13g2_stdcell_typ_1p20V_25C.lib
opt_clean

# Here, we output area for our record and then the netlist that OpenSTA needs
tee -o dse.stat.json stat -liberty sg13g2_stdcell_typ_1p20V_25C.lib -json
write_verilog dse.postmap.v
```

Q2.6 Open analysis.tcl and follow the instructions to complete it. When it's done, screenshot it and paste it here. You will commit this file later.

section2 > analysis.tcl

```
1
2   read_verilog dse.postmap.v
3   read_liberty sg13g2_stdcell_typ_1p20V_25C.lib
4   link_design asqrt_top
5
6   # TODO: Create a clock with a 5 ns period here using the syntax from Section 1
7   create_clock -name clk -period 5 {clk}
8
9   # This will write out the analysis report so Python can use it.
10  report_checks -format json > dse.analysis.json
```

Q2.7 Run the completed explore.py. A GUI graph should pop up. If it doesn't, you can also open sub/dse.png. Paste a screenshot here.



Q2.8 In 5-6 sentences, write your own analysis based on what the graph shows you.

- Can you pick a best design in terms of f_{max} or area?
- Can you pick a best design in general – perhaps by justifying some sort of balance of factors?

The highest f_max is observed at **N_CYCLES = 8**, indicating that deeper pipelines can operate at higher frequencies. However, this configuration also results in the largest area footprint. On

the other hand, the smallest area occurs at **N_CYCLES = 2**, but it comes with a noticeable drop in f_max, which may affect overall performance. A good trade-off appears at **N_CYCLES = 4**, where the f_max is only slightly lower than at 8, but the area is significantly reduced. This configuration strikes a solid balance between performance and hardware cost. Therefore, N_CYCLES = 4 could be considered the best general design point based on both efficiency and resource usage.

Q2.9 Pick any "good design" (based on what you looked at above in Q2.8) and, below, write out its N_CYCLES and provide its statistics (f_{max} and area) – you may look at `dse.table.json` for this.

N_CYCLES = 4
f_{max} = 577.03 MHz
area = 13869.6894 um^2

Q2.10 Imagine that you had done this design space sweep using the full P&R flow – running not just synthesis/STA but also place and route. Answer in 5-6 sentences:

- How much longer would it take? Write down your intuition —you don't need precise numbers, but think about how the complexity and runtime might scale compared to synthesis and STA alone. Consider what extra steps P&R introduces, and how those could impact total exploration time.
- Does this run-time have implications for how many design configurations you can test?
- Do you think that performing P&R will improve the accuracy of analysis results? Would it be worthwhile given the longer run-time?

Based on my experience, the full P&R flow for the GCD design in Lab0 takes about 1 minute and 30 seconds. Since the ASQRT design is simpler at the RTL level than GCD, we can expect the P&R runtime to be slightly shorter or roughly in the same range per configuration. However, performing a full P&R sweep across many configurations would still significantly increase the total exploration time compared to just synthesis and STA, which are much faster. This increased runtime limits the number of configurations we can practically explore, especially if we want rapid iteration. On the other hand, P&R would provide more accurate estimates of area, timing, and congestion since it reflects actual physical implementation constraints. Therefore, although time-consuming, P&R is worthwhile when high accuracy is needed for final design selection or sign-off.

It is now time to prepare your submission. Unlike other sections, there is no submission subdirectory. **Please ensure the following files are present, completed, and non-empty in `./section2`:**

- `explore.py`
- `synth.tcl`
- `analysis.tcl`
- `dse.png`
- `dse.table.json`
- `dse.postmap.v`
- `dse.stat.json`
- `dse.analysis.json`

If there are more files present, it's okay to leave them be. You may wish to commit to save your progress here.

Ensure you answered all questions in this section and ensure all the files you need for submission are in the directory before continuing to th e next section.

Section III Closing the Loop

Now that you have completed a design space exploration, it is time to validate your selected design through a full implementation. The goal of the synthesis-only DSE in Section II was to enable exploration across a wider range of configurations using the same amount of compute resources —a common and critical consideration in industry workflows.

In a more realistic setting, a team might select a top-N set of candidate designs for full-flow implementation, rather than committing to just one. However, for this lab, you will take the single best design you selected in Q2.9 and run it through the complete OpenROAD-flow-scripts flow, including placement, clock tree synthesis, and routing —thereby validating its true performance, area, and physical feasibility.

From the root of the repo, cd into `./section3`. Then, run `orfs_copy` (similar to Lab 0) to bring in an instance of OpenROAD-flow-scripts (this should not be committed later).

In ./section3, you will find `config.mk`, `constraints.sdc`, and the RTL again. You should be familiar with these files from Lab 0 – they are what drive implementation in ORFS. What we are doing here is running a custom design in ORFS without adding it to ORFS' folder structure.

These files are mostly complete, except you need to complete the `VERILOG_TOP_PARAMS` section in `config.mk`.

Q3.1 Complete `config.mk` and paste a screenshot below.

section3 > config.mk

```
export PLATFORM         = ihp-sg13g2
export DESIGN_NAME      = asqrt_top

export VERILOG_FILES    = rtl/*.v
export SDC_FILE         = constraint.sdc

export USE_FILL = 1

export CORE_UTILIZATION = 40
export PLACE_DENSITY    = 0.50
export TNS_END_PERCENT  = 100

# !Complete the following the following
# export VERILOG_TOP_PARAMS = N_CYCLES <N> N_PIPES <N> ..
export VERILOG_TOP_PARAMS = N_CYCLES 4 N_PIPES 4 N_DEPTH 4
```

Now, run ORFS using the command that follows:

```
make --file=OpenROAD-flow-scripts/flow/Makefile DESIGN_CONFIG=config.mk
```

Q3.2 When the flow has completed, you will see the familiar directories reports / logs / results / objects.

Open `logs/ihp-sg13g2/asqrt_top/base/6_report.log` and search for "`report_design_area`"

What is the area and utilization reported?

In 1-2 sentences, describe whether what you observe is similar to what the DSE process estimated, keeping in mind that the area reported by DSE is at 100% utilization?

area = 23535 u^2

utilization = 96%

The reported area is 23,535 μm² with a utilization of 96%, w hich is close to the 100% utilization assumption used during the DSE process. This suggests that the DSE's area estimation was reasonably accurate, with only a slight margin below full utilization in the actual implementation.

Now, open `section3/reports/ihp-sg13g2/asqrt_top/base/6_finish.rpt` and search for "`data arrival time`" under the "`finish report_checks -path_delay max reg to reg`" section. The positive of this number represents your minimum clock period.

You can compare this to the f_{max} you estimated in DSE using the following formula:

Given a clock period T, $f_{max} = \frac{1}{T}$ Keep in mind this will output frequency in Hz and it requires that T is defined in seconds (i.e. $1\,ns = 1 \cdot 10^{-9}\,s$).

Q3.3 What is the clock period and f_{max}?
In 1-2 sentences, how does this compare to the f_{max} you found in Q2.9?

data arrival time = -2.24 ns
f_{max} = 446.4286 MHz
The f_max obtained from the full P&R flow is 446.43 MHz, which is lower than the 577.03 MHz estimated in Q2.9 during DSE. This difference is expected, as DSE only considers synthesis and ideal timing, while P&R reflects more realistic physical delays such as routing congestion and placement impact.

Q3.4 Answer the following in 5-6 sentences:

- Based on both timing and area, do you think the DSE process gave you a realistic estimate of this design's performance?
- Was the synthesis and STA output (from Section II) generally pessimistic or optimistic compared to the full P&R results?
- Recalling your answer in Q2.10, has your response changed after seeing the P&R run-time and/or Quality-of-Results?

Based on both timing and area, the DSE process provided a reasonablyrealistic estimate of the design's performance, though it slightly overestimated the maximum frequency (f_max) compared to the full P&R results. The synthesis and STA outputs were generally optimistic because they do not fully account for physical effects like routing delays and congestion, which are reflected in the P&R results. The area estimation from DSE, assuming 100% utilization, was quite close to the final reported utilization -adjusted area after P&R. After seeing the P&R run-time and quality-of-results, my response remains consistent: while DSE is valuable for early-stage exploration due to its speed and approximate accuracy, the full P&R flow is necessary to capture realistic performance and implementation challenges. This reinforces the need to balance between fast iterative exploration and detailed physical implementation in design space exploration.

For this section, you will submit your completed `config.mk` file alongside the ORFS output directories (reports / logs / results / objects). You do not need to move them to any submission directory.

If you placed ORFS in the directory with a different directory name than the default

OpenROAD-flow-scripts, please delete it before committing.

You can now commit all the files in ./section3 and its subdirectories.

Ensure you answered all questions in this section and ensure all the files you need for submissi on are in the directory before continuing to the next section.

Finalization

Please review each section and ensure you have answered all the questions.
Ensure you have committed and pushed all files required for submission for each section.

Go to your GitHub repo and verify that the push was successful. Your push is all that is required for submission to count in GitHub and you may push until the Lab deadline.

Additional Notes

In this lab, we did not cover retiming – performance optimization performed by moving logic around register boundaries. ABC supports this however.
We also did not cover repipelining – a more general case of optimization whereby the entire pipeli ne is configured with differing numbers of stages. Repipelining in industry contexts is currently done manually and has been one of the major drivers for CPU performance increases in recent years.
Automated repipelining is a topic associated with High -Level Synthesis as HDLs like Verilog require explicit FSM controllers that are not amenable to repipelining changes that would, say, change pipeline depth or wait cycles.

<end>